How-To Library

Informatica

Performance Tuning and Sizing Guidelines for PowerExchange for Snowflake on the Spark Engine

© Copyright Informatica LLC 2019, 2021. Informatica, the Informatica logo, and Big Data Management are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html

Abstract

When you use PowerExchange for Snowflake to read data from or write data to Snowflake, multiple factors such as hardware parameters, database parameters, Hadoop cluster parameters, and Informatica mapping parameters impact the adapter performance. You can optimize the performance by tuning these parameters appropriately. This article describes general reference guidelines to help you tune the performance of PowerExchange for Snowflake on the Spark engine.

Supported Versions

- PowerExchange® for Snowflake 10.2.2

Table of Contents

Performance Tuning Overview. 2
Snowflake Overview. 2
Performance Tuning Parameters. 3
General Best Practices. 3
Tuning the Hardware. 3
Tuning the Snowflake Data Warehouse 4
Tuning the Spark Engine. 5
Optimizing the Snowflake Mapping. 6

Performance Tuning Overview

Performance tuning is an iterative process in which you analyze the performance, use guidelines to estimate and define parameters that impact the performance, and monitor and adjust the results as required.

This document describes the key hardware, database, Hadoop cluster, and Informatica mapping parameters that you can tune to optimize the performance of PowerExchange for Snowflake on the Spark engine. This document also includes case studies that involve the tuning specifications required when you run a mapping on the Spark engine to write data to Snowflake from HDFS using Amazon EMR. The graphs constructed from the case studies illustrate the impact that the tuning has on PowerExchange for Snowflake performance.

Note: The performance testing results listed in this article are based on observations in an internal Informatica environment using data from real-world scenarios. The performance of PowerExchange for Snowflake might vary based on individual environments and other parameters even when you use the same data.

Snowflake Overview

Snowflake is an analytic data warehouse provided as Software-as-a-Service (SaaS). Snowflake provides a data warehouse that is faster, easier to use, and far more flexible than traditional data warehouse offerings.

The Snowflake data warehouse is not built on an existing database or on a big data software platform such as Hadoop. The Snowflake data warehouse uses a SQL database engine with a unique architecture designed for the Cloud. Snowflake is similar to other enterprise data warehouses, but has additional functionalities and unique capabilities.

Performance Tuning Parameters

You can optimize the performance of a read or write operation using PowerExchange for Snowflake by tuning the following parameters:

- Hardware requirements for Amazon EMR cluster
- Snowflake database
- Spark engine
- Snowflake mapping

General Best Practices

To optimize the performance, consider the following best practices for Amazon EMR and Snowflake Data Warehouse:

- Host the Informatica domain on a EC2 machine to optimize the Model Repository Service. Ensure that the EC2 machine hosting the Informatica domain is in the same region as the Hadoop cluster.
- Choose the Snowflake Data Warehouse in the region where the Amazon EMR is located, or choose the Amazon EMR in the region where the Snowflake Data Warehouse is located.
- Make sure that you complete tuning the Spark engine.
- Pick the right instance size for the EC2 and Snowflake nodes.

Tuning the Hardware

To optimize the performance, consider the following tuning guidelines for your Big Data Management environment.

Instance size for Informatica Data Integration Service in an Amazon EMR cluster

When you provision an Amazon EMR Hadoop cluster instance, you must choose the instance size of your nodes appropriately as some workloads are CPU intensive while others are disk I/O or memory intensive.

Use the following guidelines to choose the right instance size for Informatica Data Integration Service in an Amazon EMR cluster:

- For memory intensive jobs, you must ensure that the `m2` family instance size have enough memory and CPU power to perform the task.
- For CPU intensive jobs, you must ensure that you have a `c2-series` instance.
- If the jobs are both memory and CPU intensive, you must ensure that you use the `cg1.4xlarge` or `cc2.8xlarge` instance as they have suitable memory and CPU power to handle a heavy workload.

Amazon EMR Hadoop cluster

The Amazon EMR Hadoop cluster configures each instances with the appropriate Hadoop configuration settings, such as Java memory size, number of mappers, and number of reducers for each instances. When you provision a Snowflake cluster, you must choose the following parameters appropriately:

- Java memory or heap size for each daemon
- Number of mappers for each instance
- Number of reducers for each instance

Choose the Snowflake cluster in the same region where the Data Integration Service machine runs. To optimize performance, use the pre-configured Amazon EMR cluster parameters even though you can change your Hadoop configuration by bootstrapping the Amazon EMR cluster.

Disk speed

Snowflake recommends that you use the `cc2`, `cg1`, or `D2` series that are ideal for the Hadoop distributed storage and MapR distributed HDD storage. You must have two disks each of 2 TB instead of two disks with 1 TB and 4 TB to reduce the disk I/O, improve write performance, and minimize the downtime.

To optimize performance, use multiple disks with smaller capacity instead of a single disk with larger capacity.

CPU clock speed

If your mappings consist of multiple CPU bound transformations in a medium or large data sets, you must consider using a high clock speed processor.

Network

If you need to shuffle or sort large amount data, you must use the `cg1.4xlarge` and `cc2.8xlarge` instances with 10 Gb of network speed. When the data is in memory, multiple applications are network bound.

To make the **Distributed Reduce** applications such as group-bys, reduce-bys, and SQL joins to perform faster, use 10 Gb or higher network. You can see how much data shuffles or sorts across the network from the **UI Monitoring Application**.

Tuning the Snowflake Data Warehouse

You can tune the Snowflake Data Warehouse components to optimize the read and write performance.

Dedicate Separate Warehouses for Snowflake Load and Query Operations

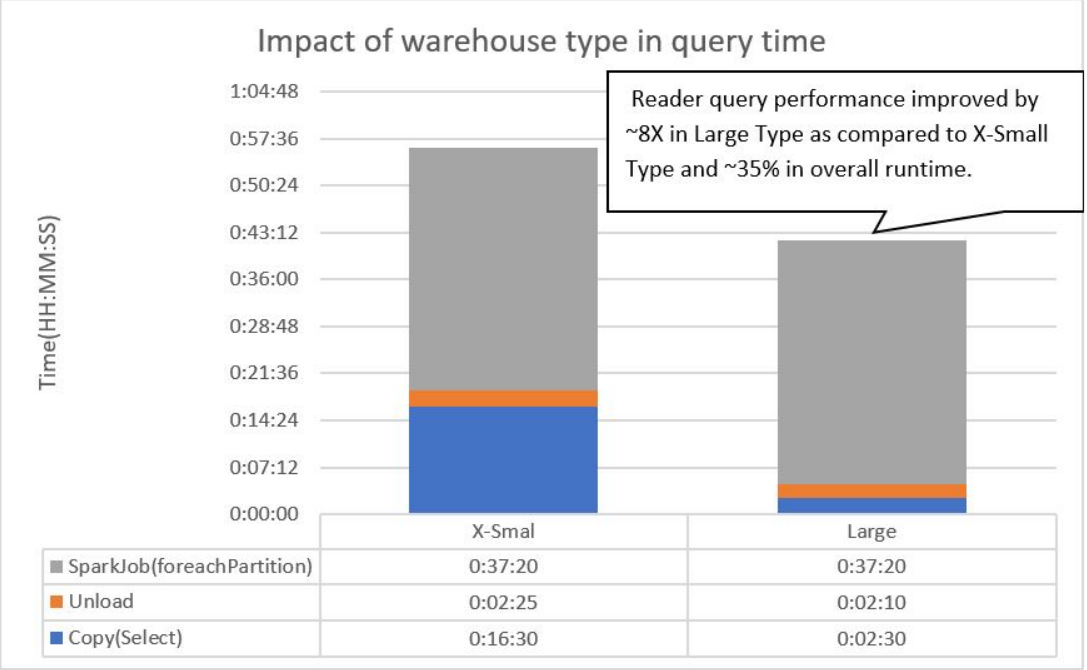
When you configure a mapping to load large data sets, the query performance can get impacted. To optimize performance when you configure load and query operations, you must dedicate separate warehouses. The number and capacity of servers in a warehouse determine the number of data files that can be processed in parallel.

You must follow the file sizing guidelines so that the data load uses minimal resources. For more information on preparing you data files, see:
https://docs.snowflake.net/manuals/user-guide/data-load-considerations-prepare.html

You can split larger data files to allow the load to scale linearly. A smaller warehouse is generally sufficient unless you want to concurrently load a large number of data ranging from a few hundreds to thousands of files. A larger warehouse such as X-Large or 2X-Large consumes more credits and might not optimize the performance.

To understand the impact of dedicating warehouses to process queries, refer to the Snowflake guidelines and make appropriate changes depending upon the complexity of data and the amount of data that you want to process.

The following image shows the impact of the warehouse type on the query performance:



Tuning the Spark Engine

You must tune the Spark engine based on your requirements. You can change the default values for the Spark engine in the advanced properties in the Spark configuration section of the Hadoop connection.

The following list describes the tuning parameters for the Spark engine:

spark.executor.memory

The amount of memory to use for each executor process. For optimized performance, use the default value. If you want to change the default value, specify the value in the following format: 2g

The recommended value is 6G for processing data without any resource crunch at runtime.

spark.executor.cores

The number of cores to use on each executor. One core for every executor is recommended for simple read and write operations.

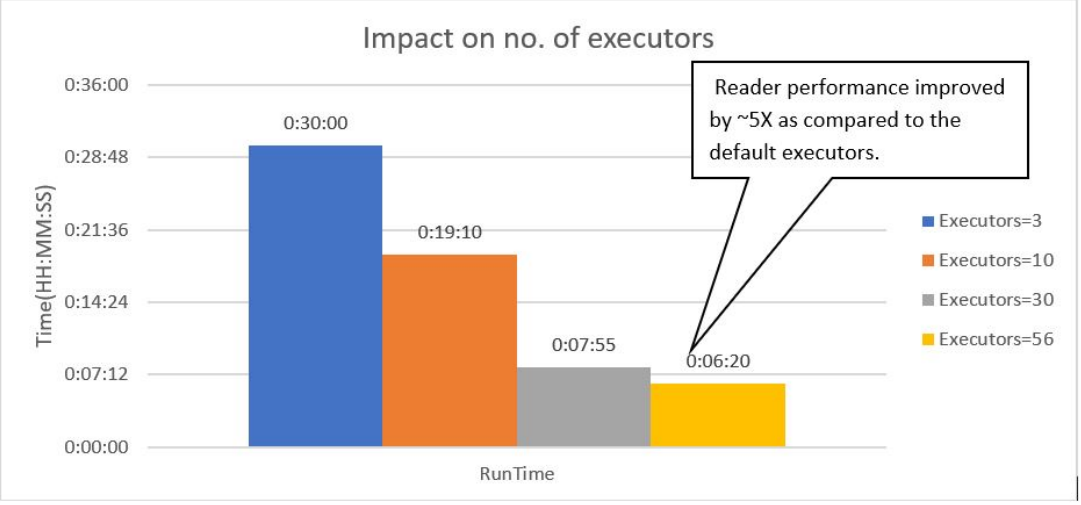
spark.executor.instances

The initial number of executors to run if dynamic allocation is enabled. The recommended number is 30 or 60 based on the cluster capacity and defined SLA.

spark.driver.memory

The amount of memory to use for the driver process where SparkContext is initialized. For optimized performance, use the default value. If you want to change the default value, specify the value in the following format: 2g

The following image shows an increase in reader performance with an increase in the number of executors:



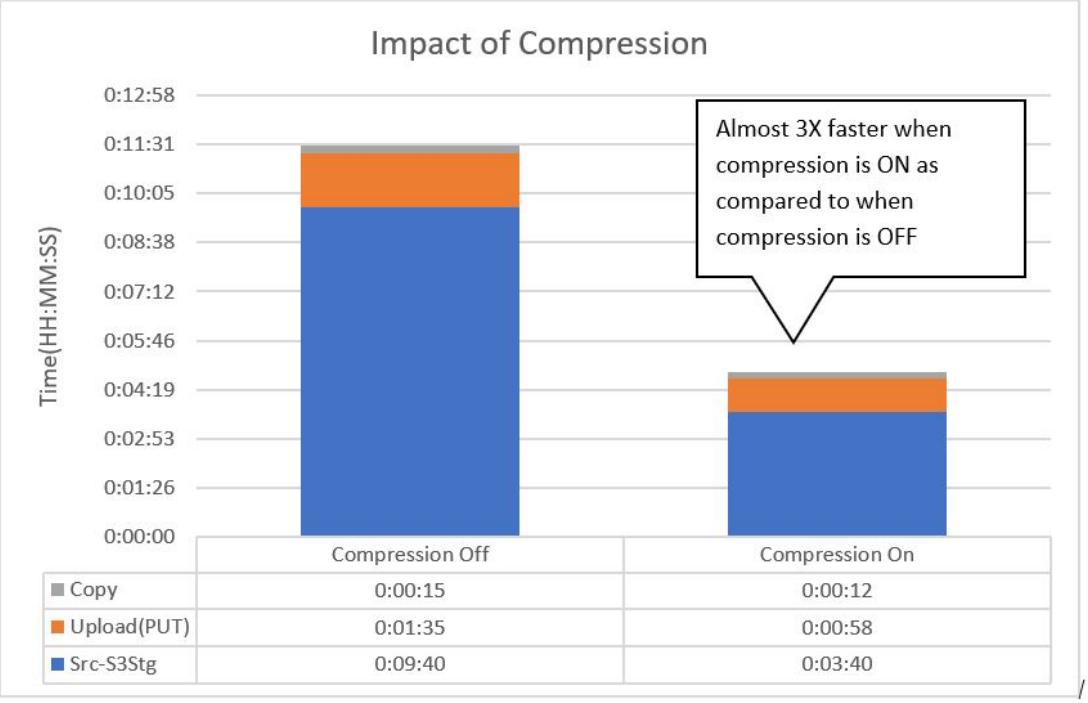
Optimizing the Snowflake Mapping

You can configure the compression parameter to optimize the Snowflake mapping performance:.

Compress Data for Snowflake Write Operations

To optimize the performance of a Snowflake write operation, you must compress the data when you run a mapping on the Spark engine. To compress files before writing to Snowflake tables, you can specify the run-time parameter `Compression=On` in the Snowflake data object write operation properties. By default, compression is on.

The following image shows the impact of compression when the Data Integration Service loads data into Snowflake:



Authors

Siddiq Hussain

Krishna Prabhakar Devarakonda

Dimple Rai